Simple Black Box Attacks on Neural Networks through Feature Extraction

Kushagra Tiwary

Advised by: Vincent Bindschaedler

Abstract

Neural Networks have gained tremendous popularity for achieving high performance in many classification tasks which has led to their use in many important applications such as self-driving cars, and medical imaging among others. These systems require the models to be highly robust and reliable. However, these networks have been shown repeatedly to be very perceptible to adversarial attacks. In this paper we investigate the response of two different network architectures on the MNIST dataset by conducting black-box adversarial attacks.

We attack the network through extracting features relevant to the classification of that particular dataset by slightly perturbing these features. Through even small perturbations targeted at these highly important areas, we achieve high classification errors. Finally, we discuss how employing these techniques during training as a form of data augmentation can make these networks more robust to susceptible attacks.

1 Introduction

Neural Networks are a certain kind of machine learning classifier that has recently gained huge popularity due to their exceptional performance in classification tasks previously deemed difficult for a machine. [source] These classifiers are being used in important applications like self-driving cars, and medical imaging which require the network to be very robust and reliable against possible adversarial attacks. [source] Many efforts in the security and machine learning communities have identified ways to expose a network's robustness and vulnerabilities to an adversarial attack.

Related work done in this area show that even with their state-of-the-art performance, these networks are highly susceptible to small changes in the inputs[1], [2]. Finding such vulnerabilities in the network not only help us understand how these networks behave but consequently also make sure that they are robust. In this paper, we show how susceptible these networks are to black-box adversarial attacks when the adversary doesn't have any access to the network except for its inputs and outputs [1]. Due to such restrictions, this attack is a more challenging attack than on one where the network's parameters are known. [source]

To define the goals of an adversarial attack, we have used a similar definition to Narodytska et al. (2016): Given an Image I which is classified correctly by the network, is it possible to construct a transformation of I by changing its pixel values in such a way so that I is then misclassified by the network. Additionally, we also seek to minimize the amount of perturbation we apply to an image in such a way that the image is still classified correctly by a human. Our proposed attacks to the network are very simple in nature, however cause a very high classification error.

2 Preliminaries

2.1 Models used

To test our adversarial image generation algorithms, we use two Image Classification networks widely used today: Deep Neural Network (DNN) and a Convolutional Neural Network (CNN). On the MNIST dataset, we use the DNN and the CNN. All of these models have reached very high

classification accuracies: 99.05% on the CNN and 94.9% on the DNN. The network architecture for both of the networks are shown in table 1 and 2.

Layer	Neurons	activation
0-input	784	N/A
1-hidden	30	N/A
2-output	10	Softmax

Table 1: Network architecture of the DNN used for MNIST

Type	Kernel Size [D, H, W]	Activation
Input	[3,28,28]	None
Convolution	[32,3,3]	Relu
Convolution	[64,3,3]	Relu
Max Pooling	[-,2,2] stride 2	None
Dropout	N/A	N/A
Dense	[128]	N/A
Dropout	N/A	N/A
Dense	[10]	Softmax

Table 2: Network architecture of the CNN used for MNIST

Since the MNIST is divided into train, validation and test data, the model was trained on the Train Data while the classification accuracies were calculated on the test data. Due to the high classification scores of both of these networks on this dataset further validation of the model was not done.

2.2 Valid-Adversarial Images and Perturbation Function

Let's define Φ to represent the neural network for an input image I and outputs L, the label of the image. Given an input image I and its label $L(I)$ we consider I to be a *valid image iff* the network classifies it correctly into $L(I)$ i.e. $\Phi(I) = L(I)$. An adversarial Image I_{adv} is defined as a *valid image* I that the network does not classify correctly after we have applied some perturbation to it, therefore for an adversarial image , $\Phi(I_{adv}) \neq L(I)$. Given this, we also define an adversarial pixel or a set of adversarial pixels to be pixels that when perturbed, causes a *valid image* I to become an adversarial image I_{adv}. The search algorithms proposed searches for adversarial pixels in these valid images in an attempt to generate more adversarial images.

Our goal of the paper is that given a trained neural network and a test dataset, we want to maximize the number of adversarial images. We will be using the classification error of the network on the test data as a metric to evaluate how successful our algorithms are in generating these adversarial images.

To maintain consistency and simulate a real world scenario, all images before and after the perturbation must preserve the range of the images that the network was originally trained on. This is important since we don't want to consider images outside the bounds of what the network has seen since we believe that in a real world scenario, inputs would be truncated to match the requirements of the network. The set of pixels outputted by the adversarial pixel search algorithm will be called adversarial pixels, P_{adv}, and they will be the target pixels the perturbation function will perturb in the original image. The network will be run again on these modified images, and the classification error will be measured and compared to the regular error.

This leads to the definition of the perturbation function to be any function P that takes in an input pixel from a valid image $I(x, y, :) \in [LB, UB]$ *such that* $P(I(x, y, :)) \in [LB, UB]$. P_{adv} is also be the output of the search algorithms. With the following constraints on what we consider a valid image, we explored many different perturbation functions. However, through experimentation, the following perturbation function gave us the largest classification errors. This function is borrowed from Narodytska et al.

$$P_1(I,\ P_{adv}) = sign(I(x, y, :)) * p \ where\ p \in [0, 1]$$

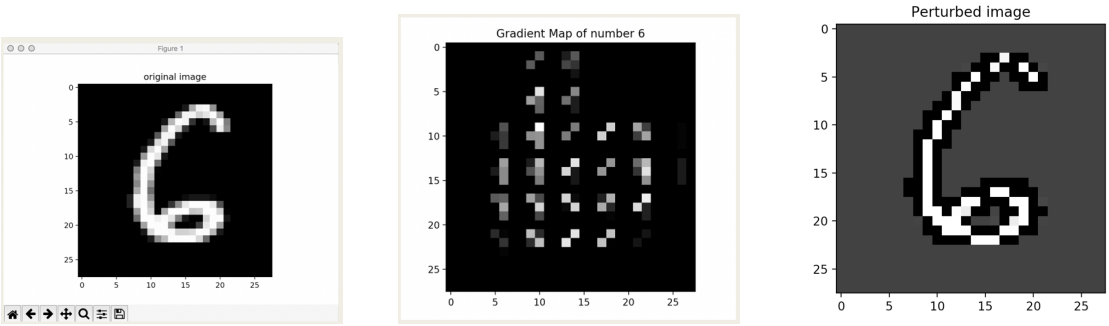

Figure 1: An example of an image mis-classified by perturbing the pixels extracted by an image gradient filter- Sobel Filter (output shown in the second image) . In this case, we perturbed all the pixels around the edges and setting the edge pixels detected by the filter to 0.1. The digit was cllassified as 5.

3 Adversarial Pixel Generation Strategies

Before the introduction of Deep Neural Networks into mainstream, computer vision for image recognition was based on hand crafted feature extractors. These hand-crafted feature extraction methods were designed to extract features such as gradients, corners, or more complex features like HOG or SIFT that were deemed to be distinct by the researcher. These features could then be used to train a classifier to recognize the image. [paper] However, with the advent of neural networks, the network now learns some of these features by itself through training over thousands of images. This end-to-end learning, without the researcher needing to extract or define the features, makes these algorithms highly scalable and requires less work to develop. [paper] Moreover, these algorithms have shown to have higher accuracies than traditional recognition systems. [paper]

3.1 Adversarial Pixels as Image features

For image recognition, Convolutional Neural Networks have shown tremendous promise accounting for the spatial features of an image to boil the image down into highly abstracted representations used to classify the image. In CNNs, the weights are in the form of several kernels which are convolved with the inputs to create activation maps at each layer. This convolution happens repeatedly and the kernels' parameters are then adjusted to minimize the loss. [paper?]

These filters/activation maps are often deemed to capture a certain feature in the image. In fact, [paper] has shown that the certain maps generally tend to be activated when a certain gradient, edge or a corner is detected across different image objects and datasets, while the deeper filters get activated by more abstract features in the image and are more closely associated with the particular dataset used. Our adversarial pixel search models are based on extracting those *general* features from an image that are perceptible to be detected by these first layer filters (gradient, edges etc.). These feature locations are then perturbed by the perturbation functions defined in section 3.

Through our work we show that by perturbing those *general* features of the image we can generate an adversarial image- get the network to mis classify images that were previously classified correctly. This means that choosing a good feature extractor based on the properties of the image dataset is important. We experiment with different feature extractors like FAST, Histogram of Gradients, and two filters: Gabor and the Sobel filters. We evaluate these methods against each other and the non-perturbed test classification error of the network- which we call *baseline error of the network*.

3.2 Image Gradients

The influence of edges and curves in the characterization of an Image or its influence in image classification and detection tasks cannot be understated. The algorithm below calculates the

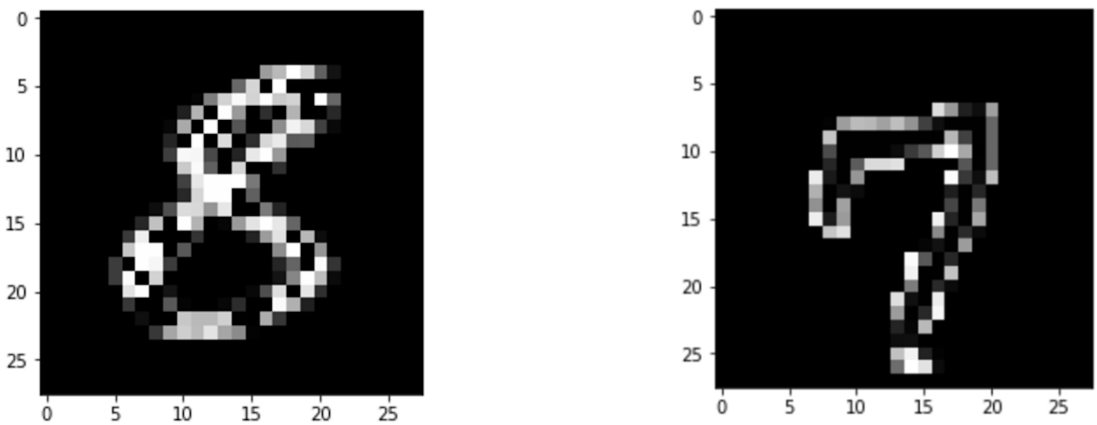

Figure 2: An example of an image mis-classified by perturbing the features detected by the Fast Corner Detector. The 8 was mis-classified as 5 and 7 as 9. This is an interesting case where the 8 and 7 are still quite clearly distinguishable to the human eye since only certain corner pixels are perturbed. This is a great example of how the local features the kernels look for can be attacked if indeed the kernel has learnt to look for the these corners as a method to bin images in the right categories. We have to be careful while training these Neural Networks as they can be learning "quick and easy" ways to classify images which can lead to it being highly susceptible to such attacks that preserve the overall image.

gradient magnitude and the angle of the image and then uses that feature map to select adversarial pixels to perturb.

The magnitude gradient of the image is calculated by convolving the Sobel filter horizontally g_x and vertically g_y with the input Image. The magnitude is calculated by $G = \sqrt{g_x^2 + g_y^2}$. An example of the output magnitude feature vector of the image is shown below in figure <insert figure>. This feature vector is then max-pooled using a 2 by 2 max filter which reduces the image's dimensionality. By calculating the argmax of the max-pooled vector we find the pixels with the greatest magnitude. We let this pixel in this max-pooled output image be the adversarial pixel. We then apply the perturbation function to the four neighboring pixels of the original (since the image had been down sampled due to the max pooling function) and produce the adversarial image.

We let k be the amount of pixels we want to perturb, P_{adv} be a $k \times 1$ vector which is the output of the adversarial algorithm, and I_{adv} to be the proposed adversarial image after the perturbation has been applied to the original Image.

$$g_x = Sobel_x\left(I\right),\ g_y = Sobel_y\left(I\right)\ \ and\ G = \sqrt{g_x^2 + g_y^2}\,.$$

$$P_{adv}^{k\times 1} = argmax\left(maxPool\left(G\right)\right),\ I_{adv} = P_1\left(P_{adv}^{k\times 1}\right).$$

3.3 Gabor Filters

Gabor filters are often used in Image processing for feature extraction and texture analysis in images. [paper] These filters are orientation-sensitive filters and they can be oriented in a particular direction to give very strong responses to an input image that has structures also oriented in that particular direction. This makes them very versatile and useful to different shapes, sizes, and smoothness of an image. Although there is some evidence that this filter is closely linked to the filter response of the human visual system, there is no empirical evidence. Additionally, initial neural network layers have also found to have responses similar to those of gabor filters.

To generate adversarial pixels we convolve a gabor filter corresponding to frequency f with the image to extract the real response of the filter. For each dataset, frequencies from a wide range

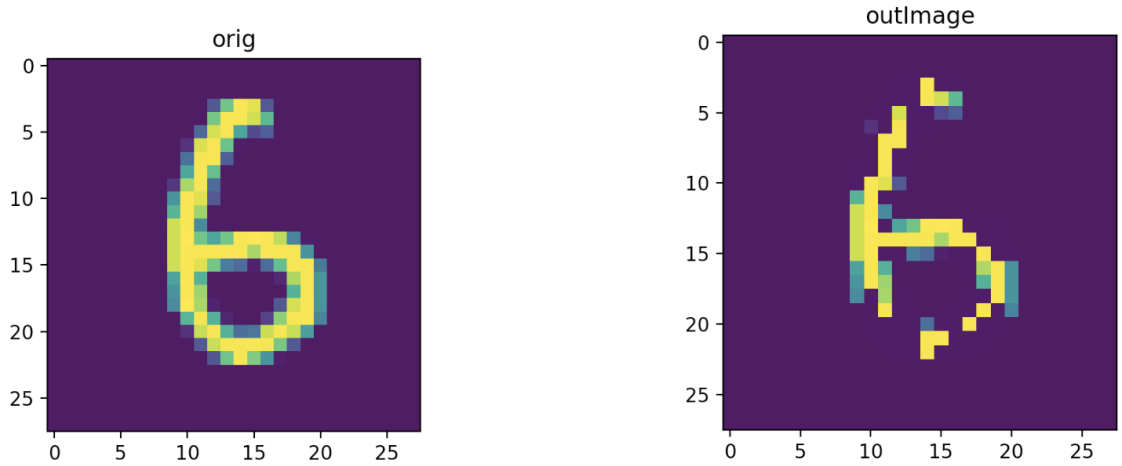

Figure 3: An example of an image mis-classified by perturbing 13% of its pixels using HOG features with the above equation is shown in figure below. The image originally classified as a 6, is now classified as a 5. Note that randomly perturbing 13% of its pixels doesn't incur a mis-classification.

are tested before one is selected which gives the largest classification error. We apply the argmax function to the real response of the filter, and also to the max pool output to get two sets of adversarial pixels. (similar to other cases)

$$P_{adv}^{k\times 1} = argmax\left(maxPool\left(Gabor\left(I,\ f\right)\right)\right) \quad with\ output\ I_{adv1} = P_1\left(P_{adv}^{k\times 1}\right).$$

3.4 Histogram Of Gradients

The Histogram of Gradients, commonly denoted as HOG, was first introduced by Dalal et al. in 2005 as a robust dense feature extraction system for pedestrian detection in images [3]. It captures information about the structures and shapes of objects in the image by leveraging image gradients. Going into the inner details of HOG is beyond the scope of the paper, and we let it up to the reader to find out more however, HOG has shown to be quite effective as a feature extraction technique for applications such as licence plates [4]. We use a scikit-image library implementation to extract the HOG features of an image. We max-pool the output as done previously and treat the pixels with the greatest magnitude to be the adversarial pixels:

$$P_{adv}^{k\times 1} = argmax\left(maxPool\left(HOG\left(I,\ f\right)\right)\right) \quad with\ output\ I_{adv} = P_1\left(P_{adv}^{k\times 1}\right).$$

Through this, we select pixels that have the greatest contribution to the "HOG" features and perturb those. An example of this is shown in Figure 3.

3.5 Fast Corner Detection

Corners are often termed as "interest points" and are places where edges change rapidly. FAST is a high-speed corner detection algorithm first introduced by Rosten and Drummond in 2006 as a fast and efficient way to detect corners in videos [5]. The FAST Algorithm is based on two user inputs: threshold value which serves as a comparison between nearby pixels and a minimum number of consecutive pixels n that should be either bright or dark- a test by the algorithm to detect a corner. Note that changing the threshold and n has a direct relation to the number of corners detected, and therefore the number of pixels perturbed. We choose the best parameter which maximizes the classification error within the bound of a maximum of k pixels being perturbed.

$$P_{adv}^{k\times 1} = FAST\left(I,\ n, threshold\right) \quad with\ output\ I_{adv1} = P_1\left(P_{adv}^{k\times 1}\right).$$

3.6 Random Perturbations

To test how well these key features extractors perform in locating an adversarial pixel, we test it against a random benchmark. In this generation strategy, we choose k pixels randomly from the

Image, and perturb it using the perturbation functions.

$$P_{adv}^{k\times 1} = Rand\,(I,\ k) \quad with\ output\ I_{adv1} = P_1\left(P_{adv}^{k\times 1}\right).$$

4 Experimentation and Results

We demonstrate the effectiveness of the above adversarial attacks in qualitatively and quantitatively below in the Tables 3, 4, and 5.

The classification errors in the table show some interesting results. We see that the Random classification error, used as a benchmark, is clearly beaten by every adversarial search algorithm proposed. Additionally the best performing algorithm is the one that uses Gabor Filters, and has been commonly known to have a filter response similar to the human eye even though this lacks empirical evidence. Note that BaseLine % error represents the classification error on the test data set without any perturbation.

Model Used (right) Feature Extraction Technique (down)	MNIST Neural Network	MNIST Convolutional NN
Base Line % Error (no perturbation)	5.09	0.950
Random % Error	5.09	1.14
Maximum Gradient	8.56	3.32
HOG Features	8.24	6.26
FAST (4.98% perturbed)	9.23	3.3
Gabor with Max Pooling	24.92	22.74
Gabor without Max Pooling	24.56	33.97

Table 3: Classification errors after perturbing 5% of the pixels in the Image

Model Used (right) Feature Extraction Technique (down)	MNIST Neural Network	MNIST Convolutional NN
BaseLine % Error (no pertubation)	5.09	0.950
Random % Error	5.27	1.43
Maximum Gradient	14.6	14.19
HOG Features	15.54	15.29
FAST (4.98% perturbed)	18.96	13.57
Gabor with Max Pooling	35.69	38.50
Gabor without Max Pooling	35.8	49.10

Table 4: Classification errors after perturbing 10% of the pixels in the Image

Model Used (right) Feature Extraction Technique (down)	MNIST Neural Network	MNIST Convolutional NN
BaseLine % Error (no pertubation)	5.09	0.950
Random % Error	6.92	2.23
Maximum Gradient	21.99	21.14
HOG Features	N/A	23.27
FAST (17.74% perturbed)	27.74	30.69
Gabor with Max Pooling	41.30	39.56
Gabor without Max Pooling	41.81	52.15

Table 5: Classification errors after perturbing 15% of the pixels in the Image

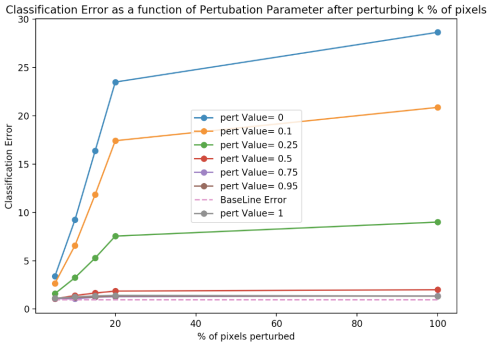

Figure 4: Graph showing how mis-classifications increase as we start to perturb more pixels in the image (using the Sobel Filter)

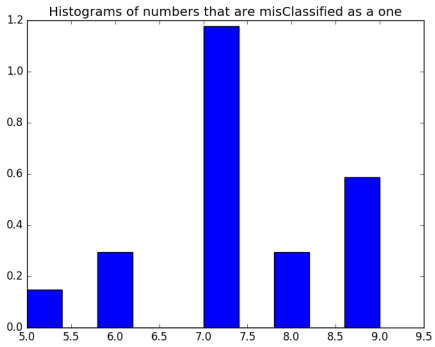

Figure 5: Histogram showing how that the number "7" is most likely to be mis-classified as a "1". We have conducted other such studies on the mis-classifications and they seem to be in line with our intuitions. 7 is likely classified as a 1, 3 as 5 etc.

5 Discussion

We showed how using simple and classical image feature extraction techniques can be used as an adversarial attack against a network by leveraging the general argument that neural networks capture general features of an image during the first few layers. This work shows a very troubling side to such networks, exposing how minor perturbations using simple algorithms can cause such high classification errors. Additionally, we also believe that such an adversarial attack can be used while training the network as well- ensuring that the network built is robust and cannot be easily fooled. We believe that we can extract features from these methods and use it as a Data Augmentation Technique. Moreover, such a threat model also serves as a second check since if the edge perturbations cause a high classification error, it means that the model is capturing edges. This will help evaluate if the model's activation maps are capturing features about the dataset, or simply over fitting- we noticed that over-fitted models tend to have high random perturbation classification errors.

6 Acknowledgements

I would like to acknowledge Vincent Bindschaedler for his constant assistance, meetings and discussions that he kept providing throughout the course of this research- without him this wouldn't be possible.

References

[1] Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z. Berkay Celik, and Ananthram Swami. Practical black-box attacks against machine learning, 2017.

[2] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. Adversarial examples in the physical world, 2017.

[3] N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. *Computer Vision and Pattern Recognition, 2005. CVPR 2005. IEEE Computer Society Conference on*, 1:886–893, 2005.

[4] Prates R.F, Cámara-Chávez G, William R Schwartz, and Menotti D. Brazilian license plate detection using histogram of oriented gradients and sliding windows. *International Journal of Computer Science and Information Technology*, 5(6):39–52, Dec 2013.

[5] Edward Rosten and Tom Drummond. Machine learning for high-speed corner detection. In Aleš Leonardis, Horst Bischof, and Axel Pinz, editors, *Computer Vision – ECCV 2006*, pages 430–443, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.